# Administrarea bazelor de date

## CURS 5: TABLE şi VARRAY

# Baza de date utilizata. Tabela STUD

| MATR | NUME | AN | GRUPA | DATAN | LOC | TUTOR | PUNCTAJ | CODS |
|---|---|---|---|---|---|---|---|---|
| 1456 | GEORGE | 4 | 1141A | 12-MAR-82 | BUCURESTI | | 2890 | 11 |
| 1325 | VASILE | 2 | 1122A | 05-OCT-84 | PITESTI | 1456 | 390 | 11 |
| 1645 | MARIA | 3 | 1131B | 17-JUN-83 | PLOIESTI | | 1400 | 11 |
| 3145 | ION | 1 | 2112B | 24-JAN-85 | PLOIESTI | 3251 | 1670 | 21 |
| 2146 | STANCA | 4 | 2141A | 15-MAY-82 | BUCURESTI | | 620 | 21 |
| 3251 | ALEX | 5 | 2153B | 07-NOV-81 | BRASOV | | 1570 | 21 |
| 2215 | ELENA | 2 | 2122A | 29-AUG-84 | BUCURESTI | 2146 | 890 | 21 |
| 4311 | ADRIAN | 3 | 2431A | 31-JUL-83 | BUCURESTI | | 450 | 24 |
| 3514 | FLOREA | 5 | 2452B | 03-FEB-81 | BRASOV | | 3230 | 24 |
| 1925 | OANA | 2 | 2421A | 20-DEC-84 | BUCURESTI | 4311 | 760 | 24 |
| 2101 | MARIUS | 1 | 2412B | 02-SEP-85 | PITESTI | 3514 | 310 | 24 |
| 4705 | VOICU | 2 | 2421B | 19-APR-84 | BRASOV | 4311 | 1290 | 24 |

# Tabelele SPEC si BURSA

| CODS | NUME | DOMENIU |
|---|---|---|
| 11 | MATEMATICA | STIINTE EXACTE |
| 21 | GEOGRAFIE | UMANIST |
| 24 | ISTORIE | UMANIST |

| TIP | PMIN | PMAX | SUMA |
|---|---|---|---|
| FARA BURSA | 0 | 399 | |
| BURSA SOCIALA | 400 | 899 | 100 |
| BURSA DE STUDIU | 900 | 1799 | 150 |
| BURSA DE MERIT | 1800 | 2499 | 200 |
| BURSA DE EXCEPTIE | 2500 | 9999 | 300 |

# Asemanari

- TABLE si VARRAY (ambele se mai numesc si colectii) se aseamănă între ele în următoarele privinţe:

1. O colecţie de acest fel poate fi stocată într-o celulă a unei tabele din baza de date (deci o tabelă poate avea o coloană de tip colecţie TABLE sau VARRAY). În cazul tablourilor asociative (TABLE .. INDEX BY) acest lucru nu era posibil.
2. Accesul la elemente se face prin indicele acestora care porneşte de la 1.
3. În ambele cazuri variabilele de acest tip pot fi iniţializate la declarare şi li se pot atribui valori în porţiunea executabilă folosind constructori de aceeaşi formă.

# Asemanari

- TABLE si VARRAY se aseamănă...

4. Numărul de elemente dintr-o astfel de colecţie poate varia pe parcursul execuţiei programului (dar în cazul VARRAY nu poate depăşi limita maximă declarată)
5. Elementele unei astfel de colecţii sunt toate de acelaşi tip de date.
6. Limitările privind tipul elementelor sunt aceleaşi în cele două cazuri.

# Deosebiri

1. Numărul maxim de elemente ale tipului TABLE nu este fixat la descrierea lui, spre deosebire de VARRAY unde se specifică această limită.
2. Indicii elementelor unui VARRAY se păstrează prin stocarea în baza de date şi regăsirea lui pe când în cazul TABLE acest lucru nu este adevărat.

- Din acest motiv tipul TABLE seamănă cu mulţimile din limbajele de programare Pascal şi C spre deosebire de tipul VARRAY care este asemănător cu tablourile.

# Deosebiri

3. În cazul unui VARRAY indicii elementelor sunt succesivi, pornind de la indicele 1, crescător cu o unitate.
- La TABLE deşi intiţial indicii elementelor sunt succesivi, prin ştergerea unor elemente pot să apară 'goluri' (operaţia de ştergere 'din mijloc' nu este posibilă la VARRAY).

3. Ca stocare fizică în baza de date, datele de tip VARRAY se memorează fie în interiorul tabelei (pentru dimensiune mai mică de 4 KB) fie în spaţiul asociat tabelei (tablespace).
4. În cazul TABLE stocarea valorilor se face într-o aşa numită 'tabelă de stocare' (store table) asociată tabelei de bază.

# Limitari

- În cazul declarării lor în PL/SQL tipul elementelor poate fi oricare tip valid PL/SQL cu excepţia REF CURSOR
- În cazul folosirii lor în SQL, pe lângă REF CURSOR sunt interzise şi tipurile:
  - BINARY_INTEGER, PLS_INTEGER
  - BOOLEAN
  - LONG, LONG RAW
  - NATURAL, NATURALN
  - POSITIVE, POSITIVEN
  - REF CURSOR
  - SIGNTYPE
  - STRING

# Declarare

- Sintaxa definiţiei acestor tipuri este următoarea:
  - TABLE:

```
TYPE nume_tip IS TABLE OF tip_element [NOT NULL];
```

  - VARRAY:

```
TYPE nume_tip IS {VARRAY | VARYING ARRAY}
(indice_maxim) OF tip_element [NOT NULL];
```

unde:

- **nume_tip** - Numele tipului definit de acea declaraţie
- **indice_maxim** - Dimensiunea maximă pentru acel VARRAY
- **tip_element** - tipul elementelor colecţiei
- **NOT NULL** - specifică opţional că elementele nu pot fi nule

# Exemple

- TABLE: Declararea a două colecţii care pot conţine un număr nedefinit de denumiri de cursuri la care sunt înscrişi doi studenţi:

```
TYPE ListaCursuri IS TABLE OF
  VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
```

# Exemple

- VARRAY: Declararea a două colecţii care pot conţine maxim 25 de denumiri de cursuri la care sunt înscrişi doi studenţi:

```
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
```

- Se observă că se poate folosi construcţia %TYPE pentru a defini o variabilă colecţie ca având acelaşi tip cu altă variabilă colecţie.

# %TYPE şi %ROWTYPE

- Pentru tipul elementelor se pot folosi de asemenea construcţiile %TYPE şi %ROWTYPE:

```
TYPE NumeStud IS TABLE OF stud.nume%TYPE;
TYPE ListaStud IS TABLE OF stud%ROWTYPE;
CURSOR c_spec IS SELECT * FROM spec;
TYPE ListaSpec IS VARRAY(20) OF
  c_spec%ROWTYPE;
```

- În primul caz, NumeStud este tipul unor colecţii de valori scalare de acelaşi tip cu coloana NUME din tabela STUD.
- În celelalte două cazuri ListaStud şi ListaSpec sunt colecţii având ca elemente înregistrări definite pe baza structurii tabelei stud respectiv a cursorului c_spec.

# Elemente de tip înregistrare

- Elementele unor colecţii de acest tip pot fi şi înregistrări definite de utilizator:

```
DECLARE
TYPE Student IS RECORD (
  nume VARCHAR2(40),
  varsta DATE);
TYPE grupa IS VARRAY(25) OF Student;
TYPE seria IS TABLE OF Student;
```

# Iniţializarea la declarare

- Iniţializarea unor colecţii de acest tip se poate face la declarare prin construcţia:

```
:= nume_tip_colectie(lista_de_valori)
```

- Exemple:

```
TYPE ListaCursuri IS TABLE OF
  VARCHAR2(30);
ListaMea ListaCursuri :=
  ListaCursuri('Analiza', 'Algebra',
  'Fizica', 'Limba engleza');
```

# Iniţializarea la declarare

- sau similar în cazul VARRAY:

```
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri :=
  ListaCursuri('Analiza', 'Algebra',
  'Fizica', 'Limba engleza');
```

# Iniţializarea la declarare

- În cazul în care constructorul a fost apelat fără valori între paranteze se generează o colecţie ***vidă dar nenulă***:

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri := ListaCursuri();
BEGIN
   IF ListaMea IS NOT NULL THEN
     .   .   .   -- condiţia adevarata!
```

# Iniţializarea la declarare

- În schimb dacă o colecţie nu este iniţializată testul de NOT NULL returnează FALSE (deci IS NULL returnează TRUE):

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
   IF ListaMea IS NOT NULL THEN
     .   .   .   -- conditie falsa
```

# Atribuirea

- Se poate lăsa neiniţializată colecţia la declarare şi iniţializa în zona executabilă. Exemplul următor este doar pentru TABLE dar similar se procedează pentru VARRAY:
- Exemplu:

```
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza',
        'Algebra', 'Fizica', 'Limba
 engleza');
```

# Atribuirea

- Lista de valori poate conţine valori nule:

```
TYPE ListaCursuri IS VARRAY(25) OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL,
        'Algebra', NULL, 'Fizica', 'Limba
  engleza');
```

- În ultimele două exemple au fost asignate colecţii conţinând 4 respectiv 7 elemente (au fost numărate şi valorile NULL).

# Atribuiri intre colectii

- Două colecţii **de acelaşi tip** se pot atribui una alteia. Dacă însă tipul diferă rezultă o excepţie.

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
TYPE AltTip IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
ListaEi  AltTip;
BEGIN
  ListaLui := ListaCursuri('Analiza', NULL, NULL,
         'Algebra', NULL, 'Fizica', 'Limba engleza');
  ListaMea := ListaLui;
  ListaEi := ListaMea; -- va genera exceptie
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
END;
```

# Compararea colecţiilor

- O colecţie poate fi testată dacă este nulă sau nu cu IS NULL şi IS NOT NULL.
- În schimb nu putem folosi = sau <> pentru a testa egalitatea sau inegalitatea a două colecţii.
- Din acest motiv o coloană care este de tip colecţie nu poate apărea în cereri SQL în conjuncţie cu clauzele DISTINCT, GROUP BY sau ORDER BY.

# Metode pentru TABLE şi VARRAY

- În manipularea datelor de tip TABLE şi VARRAY putem folosi metodele menţionate anterior: EXISTS, COUNT, FIRST, LAST, PRIOR, NEXT şi DELETE.
- Metoda DELETE nu se poate aplica la VARRAY. Se foloseşte TRIM pentru a şterge elemente de la sfârşitul tabloului.

# Metode pentru TABLE şi VARRAY

- În plus se pot folosi:

| | |
|---|---|
| **LIMIT** | **Doar pentru VARRAY: numărul maxim de elemente permis. Pentru celelalte tipuri întoarce NULL.** |
| **EXTEND[(n[,i)]]** | **Extinde tabloul cu un element nul, *n* elemente nule sau n elemente egale cu elementul cu indice *i*.** |
| **TRIM[(n)]** | **Şterge un element, respectiv *n* elemente de la sfârşitul tabloului.** |

# Exemplu: Table

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL, 'Algebra', NULL,
  'Fizica', 'Limba engleza');
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 7
  ListaMea.Extend(3, 2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 10
  ListaMea.Delete(7);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 9
  ListaMea.Delete(3, 6);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 5
  ListaMea.Trim(2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 3
END;
```

# Exemplu: VARRAY

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL,
         'Algebra', NULL, 'Fizica', 'Limba engleza');
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
  -- 7
  ListaMea.Extend(3, 2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
  -- 10
  ListaMea.Trim(2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count);
  -- 3
END;
```

# Exemple

- Metodele FIRST, LAST, NEXT şi PRIOR ne permit parcurgerea unei variabile de tip TABLE sau VARRAY.
- În unele exemple de mai jos am folosit colecţii TABLE cu elemente şterse.

# Exemple

- Exemplul 1: Parcurgerea unei colecţii cu ciclu FOR. Acest bloc ridică excepţie dacă există elemente şterse.

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
i NUMBER;
BEGIN
  ListaMea := ListaCursuri('Analiza', 'Algebra', 'Sport',
 'Fizica', 'Limba engleza', 'Programare');
  FOR i IN ListaMea.FIRST .. ListaMea.LAST LOOP
    dbms_output.put_line('Curs: '||i|| ' '||ListaMea(i));
  END LOOP;
END;
```

# Exemple

- Exemplul 1: Parcurgerea unei colecţii cu ciclu FOR. Acest bloc ridică excepţie dacă există elemente şterse.

```
Curs: 1 Analiza
Curs: 2 Algebra
Curs: 3 Sport
Curs: 4 Fizica
Curs: 5 Limba engleza
Curs: 6 Programare
```

# Exemple

- Exemplul 2: Parcurgere folosind LOOP pentru a evita excepţia menţionată anterior:

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
i NUMBER;
BEGIN
  ListaMea := ListaCursuri('Analiza', 'Algebra', 'Sport',  'Fizica',
   'Limba engleza', 'Programare');
  ListaMea.Delete(3,4);
  i := ListaMea.FIRST;
  LOOP
    dbms_output.put_line('Curs: '||i||' '||ListaMea(i));
    i := ListaMea.NEXT(i);
    EXIT WHEN i IS NULL;
  END LOOP;
END;
```

# Exemple

- Rezultat:

```
Curs: 1 Analiza
Curs: 2 Algebra
Curs: 5 Limba engleza
Curs: 6 Programare
```